COUNTER PIZZA – ESPECIFICACIONES TÉCNICAS

SISTEMA TÁCTICO DE GESTIÓN Y DESPACHO DE PEDIDOS EN EL MAPA MIRAGE

1. CONTEXTO, MOTIVACIÓN Y METODOLOGÍA DE TRABAJO

Counter Pizza es un sistema informático de simulación logística y atención comercial desarrollado en Java Swing. Su propósito fundamental es fusionar la gestión operativa de una pizzería comercial con la topología de navegación del mapa táctico *Mirage* de Counter-Strike, transformando el estudio de estructuras de datos complejas en una experiencia interactiva, estimulante y visualmente inmersiva.

> **Justificación de Trabajo Individual y Gestión del Tiempo:**
> El desarrollo del proyecto fue asumido de manera estrictamente individual por decisión propia del autor, David Bocanegra González. Debido a compromisos laborales con horarios exigentes, la modalidad individual permitió avanzar a un ritmo autónomo en horas de la noche sin entorpecer ni depender de la dinámica de un equipo de trabajo. Ante inasistencias a clases magistrales, el proceso se sustentó en el estudio independiente de grabaciones, diapositivas del curso y en la asistencia de modelos de lenguaje (LLM) para estructurar el código en Java Swing, resolver errores de sintaxis y diseñar componentes de interfaz gráfica.

2. ARQUITECTURA DE DATOS Y CUMPLIMIENTO DE LA RÚBRICA

El sistema satisface rigurosamente los requerimientos académicos al implementar **cinco estructuras de datos distintas**, clasificadas en lineales, jerárquicas y redes de grafos. Cada estructura cumple una función operativa indispensable en el flujo del software:

Estructura	Tipo / Clase Java	Rol Operativo en Counter Pizza	Complejidad / Beneficio
1. Grafo Ponderado	Red de Redes GrafoMapa.java	Representa la topología de pasillos de Mirage. Maneja nodos tácticos (T Spawn, A/B Site, Mid, TV) y aristas ponderadas por tiempo en segundos.	Modelado de mapa real. Permite recalcular rutas dinámicas.
2. Cola FIFO	Lineal Queue<Pedido>	Organiza la fila de despacho de clientes por orden estricto de llegada (*First-In, First-Out*).	Garantiza equidad comercial en la atención de pedidos. O(1).
3. Pila LIFO	Lineal Stack / ArrayDeque	Almacena las selecciones de ingredientes en el formulario para ejecutar la función **'Deshacer' (Undo)**.	Permite revertir acciones del usuario paso a paso. O(1).

Estructura	Tipo / Clase Java	Rol Operativo en Counter Pizza	Complejidad / Beneficio
4. Árbol Trie	Jerárquica / Árbol ArbolTrie.java	Alimenta la barra de búsqueda interactiva con autocompletado en tiempo real para pizzas y adicionales.	Búsqueda por prefijo altamente eficiente en tiempo O(L) donde L es longitud.
5. Cola de Prioridad / MinHeap	Jerárquica / Árbol PriorityQueue	Sustenta el algoritmo de **Dijkstra** para extraer siempre el nodo con menor tiempo acumulado.	Optimiza el cálculo de rutas óptimas a tiempo O((V + E) log V).

3. LÓGICA TÁCTICA: DIJKSTRA, BLOQUEOS Y WAYPOINTS INVISIBLES

3.1 Algoritmo de Dijkstra y Desvío por Enemigos:
El cálculo de la ruta óptima utiliza el algoritmo de Dijkstra. Cuando la aplicación genera un enemigo en un pasillo (por ejemplo, entre *T Spawn* y *TV*), esa arista específica se marca como bloqueada. Si la ruta directa queda inhabilitada, Dijkstra evalúa automáticamente las alternativas bordeando los sectores despejados, desviando al repartidor por *A Ramp* / *Palace*, cruzando *A Site* y *Connector* / *Jungle* para llegar a *Mid* o *B Site*.

3.2 Técnica de Waypoints / Nodos Invisibles (WP_):
Para evitar que los trazos rectilíneos del grafo atraviesen las paredes del plano de madera de mapa.png, se crearon nodos de paso (ej. WP_T_Ramp, WP_Conn_A). En la interfaz gráfica (PanelMapaMirage.java), el método esPuntoDePaso() identifica los nodos con prefijo WP_ y omite el renderizado de círculos y etiquetas de texto. De este modo, la ruta dorada dobla en las esquinas de los pasillos de forma fluida y natural.

4. INTERFAZ GRÁFICA RESPONSIVA Y PERSISTENCIA NOSQL JSON

4.1 Mapeo Porcentual Responsivo (0.0 a 1.0):
Para garantizar que los nodos y rutas permanezcan fijas sobre los pasillos del diorama en madera independientemente del tamaño o resolución de pantalla, las posiciones se definieron en coordenadas relativas porcentuales (clase PuntoRelativo). En cada evento de redimensionamiento de ventana, las coordenadas de cada nodo se recalculan mediante (rx * getWidth(), ry * getHeight()).

4.2 Persistencia de Transacciones (GestorPersistencia.java):
Dando cumplimiento al requisito de persistencia de datos, el sistema implementa una arquitectura basada en documentos JSON (historial_pedidos.json), equivalente conceptualmente a un almacenamiento NoSQL. Cada despacho procesado genera una transacción completa con ID único, cliente, detalle de la pizza, ingredientes, costo total, tiempo estimado de entrega en segundos y la secuencia de ruta óptima calculada por Dijkstra.

> **Ejemplo de Registro JSON en Disco (historial_pedidos.json):**
> { "idPedido": "ORD-172930214", "cliente": "David Bocanegra", "destino": "B Site", "pizza": "Pizza Pepperoni Especial + Extra Queso", "precioTotal": 18.50, "tiempoEntregaSegundos": 12, "rutaCalculada": ["T Spawn", "A Ramp", "A Site", "Jungle", "CT Spawn", "B Site"] }

5. FLUJO OPERATIVO Y MANUAL DE INTERACCIÓN

A continuación se describe la secuencia interactiva del usuario al operar la interfaz de Counter Pizza:

- **Búsqueda e Ingreso Asistido (Árbol Trie):** El usuario escribe en la barra de búsqueda. El motor Trie filtra instantáneamente los nombres de pizzas e ingredientes adicionales disponibles.
- **Personalización y Precio Dinámico:** Al seleccionar tamaño, tipo de masa e ingredientes adicionales, el sistema calcula en tiempo real el precio total en dólares.
- **Control de Edición (Pila LIFO):** Si el usuario comete un error en la selección de toppings, puede presionar el botón *'Deshacer'* para revertir secuencialmente el último ingrediente añadido.
- **Simulación de Amenaza Táctica:** El operador puede pulsar *'Generar Enemigo'*. El sistema selecciona un pasillo al azar, dibuja un emblema de alerta roja y bloquea el tramo en el grafo.
- **Encolamiento y Despacho (Cola FIFO + Dijkstra):** Al presionar *'Tomar Pedido'*, la solicitud ingresa a la fila FIFO. Al presionar *'Despachar Pedido'*, se desencadena el cálculo de Dijkstra, se traza la ruta dorada sobre el plano de madera, se muestra el tiempo en segundos y se guarda la transacción en el archivo JSON.

6. CONCLUSIONES Y VALORACIÓN ACADÉMICA

1. **Integración Conceptual Exitosa:** El proyecto demostró que las estructuras de datos avanzadas (Grafos, Tries, Heaps, Colas y Pilas) no son conceptos teóricos aislados, sino componentes sinérgicos capaces de resolver problemas complejos de logística e interfaz gráfica en una aplicación unificada.

2. **Autonomía y Aprendizaje Experiencial:** A pesar de las restricciones de tiempo laboral y los retos iniciales en la sintaxis de Java Swing, la combinación de estudio independiente y el uso metodológico de herramientas de IA permitió alcanzar un producto final sólido, funcional y libre de errores de compilación.

3. **Satisfacción y Proyección:** El resultado final supera el estándar de un trabajo académico convencional, ofreciendo un software didáctico, interactivo y con un acabado profesional del cual el autor se siente profundamente orgulloso.